SetGram: An N-gram Memory Keyed by Sets of Words Recalls Facts Asked in New Words

with a survey of memory in language models

Vuk Rosić

Abstract

Language models increasingly carry a memory beside their weights: a table looked up from the last few tokens. DeepSeek's Engram, the NanoGPT speedrun record's n-gram hints, infini-gram, kNN-LM and Memory Grafting all look up the last tokens in their exact order. We survey these methods and point out what they share: a fact stored as "N was born in Paris" cannot be found by the question "Which city was N born in?", because the words come in a different order. We propose SetGram, an n-gram table keyed by the sorted set of the last few rare words within a short window, in any order, which stores the most common next token as the exact table does. On 400 made-up biographies mixed into FineWeb text, small Muon-trained models with SetGram hints answered 81% of questions in a never-seen wording about people they trained on and 80% about people that appear only in the table, against 3% to 7% (chance about 4.5%) for exact n-gram hints, an Engram-style hashed n-gram memory, CoCoNTs-style n-gram targets, a memory layer with product keys, and infini-gram-style mixing of an exact table at test time. Mixing the SetGram table into the output at test time reached 96% and lowered web loss from 7.807 to 7.670. The recall grew from 50% to 80% over 4 times more steps and from 67% to 79% over 4 times more weights. On general web text SetGram adds little on top of a full exact table (0.003 to 0.010 loss), so it complements exact keys rather than replacing them. In the second part we study the exact hints of the record: a token's own count leaks its answer, and a full table in training stops the model from memorizing and from learning comparison; we test leave-one-out, a small training table and a taught switch as fixes, and measure hints on a finished 2.5-billion-parameter model. All models except the finished one are small, and each setting ran once. All code, including every baseline, is public [23].

1 Introduction

A language model stores facts in its weights, and it learns a fact well only after seeing it many times [15]. A growing line of work gives the model a second memory: a large table, built from text, that the model reads at each position. Engram adds hashed n-gram embeddings to DeepSeek-style models [7]; the NanoGPT speedrun record adds the most common next token after the last n tokens of a corpus and cut the record from 39.9 s to 6.7 s [1, 2]; infini-gram mixes a trillion-token n-gram table into a model's output [30]; kNN-LM retrieves stored hidden states [19].

These tables share one design choice: the key is the last tokens in their exact order. That choice is natural for predicting text, because a phrase that repeats word for word is the commonest thing a table can catch. It is a poor choice for facts. People ask about a fact in many wordings, and studies of knowledge in language models find that a model can answer questions about a fact only if it saw the fact in several wordings during training [32], and that reworded copies of a fact are retained better than exact repeats [33]. An exact table never sees the question's wording, so it cannot help.

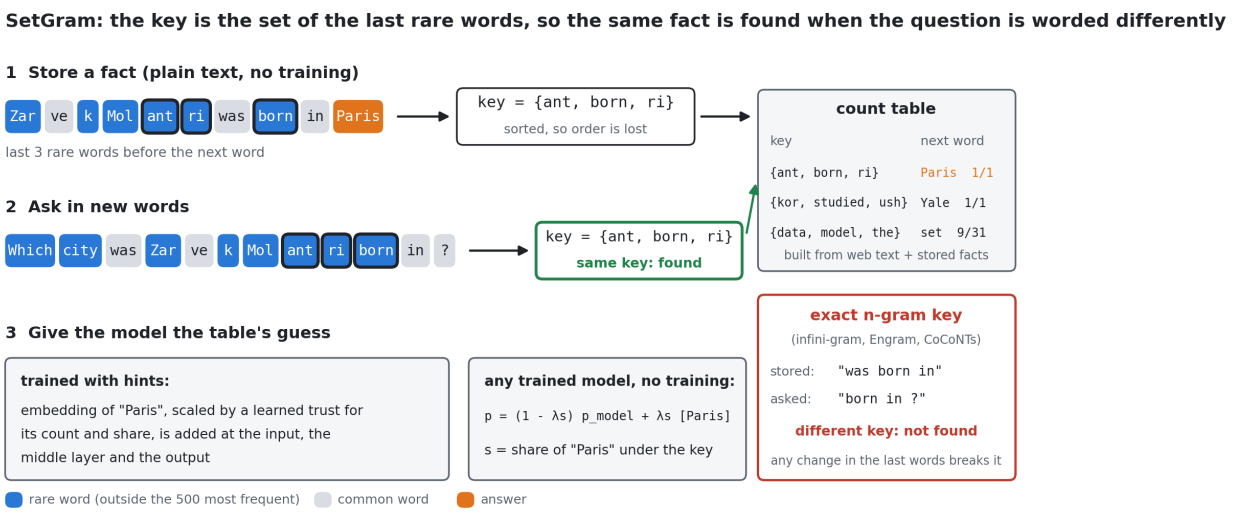


Figure 1: How SetGram works. (1) A fact is stored as plain text: for each next word, the key is the sorted set of the last 3 rare words (rare = outside the 500 most frequent tokens) in the 16 tokens before it, and the table counts which word follows. (2) A question in new words has the same rare words, so it gives the same key and finds "Paris". (3) The table's guess reaches the model either as a hint trained into the model (added at the input, the middle layer and the output) or by mixing it into the output of any trained model. Right: an exact n-gram key, as used by infini-gram, Engram and CoCoNTs, changes when the last words change, so the fact is not found.

We ask whether a table keyed by the set of rare words, with order ignored, recovers these facts, and how it compares with the published kinds of memory. We make four contributions.

1. A survey of memory in language models, organized by what is stored, how it is keyed and where it enters the model (Section 2). Every method we found keys a stored corpus by the exact order of the last tokens or by a learned vector.

2. SetGram, a next-token table keyed by the sorted set of the last few rare words in a short window (Section 3). It changes only the key of the record's hints, so it reuses their training, trust and leave-one-out.

3. A controlled test of reworded fact recall with web text loss measured on the same models, against simplified versions of five memory designs, all released as code (Section 4). SetGram recalls 80% to 96% of reworded facts where every other method stays at chance.

4. A study of exact hints in training: leakage, memorizing, reasoning, and a finished model (Part II).

Part I
Memory in Language Models, and SetGram

2 A Survey of Memory in Language Models

We group memories by what they store. Table 1 lists the methods with their key.

Count tables of next tokens. The oldest memory is the n-gram language model: counts of which token followed each run of $n-1$ tokens, smoothed across lengths [35]. Infini-gram [30] scales this to trillions of tokens with a suffix array, finds the longest exact suffix of the context that occurs in the corpus, and interpolates its next-token distribution with a neural model's output. The NanoGPT record [1, 2] stores, for suffixes of 6, 10 and 20 tokens, the most common next token and its count. That token enters the model as a hint embedding at the input, the middle and the output, weighted by a trust learned per count and share bucket, during training as well as at test. CoCoNTs [31] uses the same counts as a training target instead: the model is trained toward a mix of the true token and the corpus next-token distribution, and sees no table at inference.

Learned tables keyed by n-grams. Engram [7] hashes the last 2 and 3 tokens into large embedding tables, compresses case and accent variants first, and adds the looked-up vectors to the hidden state through a gate that reads the hidden state. It reports gains on knowledge and reasoning benchmarks at the scale of DeepSeek models. Follow-up work shares latents across n-gram embeddings [8], makes the latent codes discrete [9], factorizes the tables [41], or extends the keys [42]. TF-Engram [40] builds phrase tables without training and serves them from SSD. Earlier, N-Grammer [43] added embeddings of bigram IDs over product-quantized latents, and the Over-tokenized Transformer [44] and SCONE [45] add embeddings for multi-token n-grams to the input vocabulary. All of these key by the exact last tokens.

Learned key-value memories. Product-key memory [46] replaces a feed-forward layer with a large table of learned values selected by two half-keys; Memory Layers at Scale [47] trains such layers in billion-parameter models and reports large gains on factual question answering. The key is a learned query from the hidden state, so the memory is not tied to a token order, but it holds only what training put into it: a fact the model never trained on cannot be in it.

Stored hidden states. kNN-LM [19] stores the hidden state at every position of a corpus with its next token, and at test time mixes the next tokens of the nearest stored states into the output. Memorizing Transformers [48] store keys and values of earlier context for attention. Memory Grafting [39] finds the longest exact suffix match in a corpus and inserts the frozen hidden states stored there. The keys of kNN-LM are learned vectors, so a reworded context can in principle land near the stored one, but only if the model already maps both wordings to nearby states.

Retrieved text. RETRO [10] retrieves whole chunks of text by learned embeddings and reads them with cross-attention. In-context retrieval-augmented models put retrieved passages in the prompt, and a bag-of-words retriever such as BM25 works well there [49]. Bag-of-words retrieval ignores order, so these systems answer reworded questions, but they pay with extra prompt tokens and a search step at every query, and they are not part of training.

Order-free memory without a corpus. Cache language models [37] raise the probability of words seen recently, whatever their order, and trigger models [36] let any earlier word in the history predict the next word, through counted word pairs. NGM [38] averages the embeddings of the last n tokens without training, which also ignores order, but it stores no corpus and no next tokens.

How models acquire facts. Facts are recalled in proportion to how often they appear in pretraining [15]. A model trained on biographies in one wording memorizes them but cannot answer questions in another wording unless its training data held several wordings [32]; models trained on

"A is B" do not learn "B is A" [34]. During pretraining, reworded copies of a fact are kept longer than exact repeats [33]. These results point to wording as the main obstacle between a stored fact and a question.

Method	Stores	Key	Order of key	New facts without retraining
n-gram LM [35], infini-gram [30]	next-token counts	last tokens	exact	yes
NanoGPT record hints [2]	top next token	last 6–20 tokens	exact	yes
CoCoNTs [31]	next-token counts	last tokens	exact	no (training target)
Engram [7] and follow-ups	learned vectors	hashed last 2–3 tokens	exact	no
N-Grammer, Over-tokenized, SCONE	learned vectors	n-gram IDs	exact	no
Memory Grafting [39]	frozen hidden states	longest suffix match	exact	yes
kNN-LM [19]	hidden states, next token	learned vector	learned	yes
Memory layers [46, 47]	learned values	learned query	learned	no
RETRO [10], in-context RAG [49]	text chunks	learned or bag of words	varies	yes, via the prompt
Cache LM [37], triggers [36]	word counts	single history words	none	recent text only
NGM [38]	nothing (mean embedding)	last n embeddings	none	no
SetGram (ours)	**top next token**	**set of last rare words**	**none**	**yes**

Table 1: Memory in language models by what is stored and how it is keyed.

What is missing. The methods that store a corpus and look it up inside the model key by exact order, and the methods that ignore order either store no corpus (NGM, cache) or put retrieved text in the prompt (bag-of-words RAG). We found no memory that keys a corpus's next tokens by an unordered set of words. Our searches covered the Engram line, n-gram language model variants, kNN-LM and retrieval work, and the names SetGram and BagGram, which we found unused.

3 SetGram

Key. Let V_{common} be the 500 most frequent tokens of the corpus; every other token is *rare*. To predict token x_{t+1}, SetGram takes the last n rare tokens among $x_{t-15}, \ldots, x_t$, sorts them, and hashes the sorted list together with n and a flag that says whether x_t itself is rare:

$$k_n(t) = \text{hash}\big(n,\ [x_t \notin V_{\text{common}}],\ \text{sort}(r_1, \ldots, r_n)\big). \tag{1}$$

If fewer than n rare tokens lie in the window, there is no key. We use $n = 2, 3, 4$. The flag separates positions right after a name ("N ...") from positions after a common word ("born in ..."), which predict different things.

Value. As in the record, the table stores the most common next token after the key, its count and the total count, and a training position's own occurrence is subtracted from both (leave-one-out). The count falls into one of 7 buckets and the share of the top token into one of 6, giving 42 kinds per key length.

Use in training. The hint enters exactly as in the record:

$$H_t = \sum_n \big(\sigma_{k(n,t)}\, E[\text{hint}_n(t)] + \beta_{k(n,t)}\big), \tag{2}$$

added after the embedding, after the middle layer and before the output head, each with a learned scale. SetGram levels can sit beside exact levels in the same sum.

Use at test time only. A model trained without hints can use the table as infini-gram does: $p = (1 - \lambda s)\, p_{\text{model}} + \lambda s \mathbb{1}[\text{hint}]$, where s is the share of the longest available hint and λ is chosen on web validation loss from $\{0.1, 0.3, 0.5, 0.9\}$.

Example. The stored sentence "N was born in Paris" gives, at the position before "Paris", the rare-word set {N, born} ("was" and "in" are common) with the flag off. The test question "Which city was N born in?" gives the same set and flag before the answer, so the stored next token "Paris" returns. An exact key of any length differs: "born in" is shared by every person, and longer keys contain "N was" in one wording and "N born" in the other.

Cost. Building the keys is one pass over the corpus with a sliding buffer of rare tokens, and the table has the same layout as an exact table. Building exact and SetGram tables for 25M tokens took 27 s and 3 GB in NumPy.

4 Results: Reworded Facts

4.1 Setup

Biographies in web text. We made up 400 people with syllable names. Each has a birth city (30 choices), a university (17) and a company (28), written in four wordings, for example "N was born in A." (wording 0), "The city where N was born is A." (1), "Where was N born? A." (2) and "Which city was N born in? A." (3). Wording 0 of all 400 people is stored in the table. Wordings 1 and 2 of people 0 to 199 are training text; with `-once` each of them gets only one of these, so their facts appear in the table twice, like a rare web fact. People 200 to 399 appear only in the table. Wording 3 is the test and is never stored or trained. The table also holds the first 25M FineWeb training tokens.

Model and training. Width 32, 2 layers, 4 heads, RoPE, QK-norm, SwiGLU, RMSNorm and no biases; Muon on the hidden matrices and Adam on embeddings and memory tables. Each batch holds 6 FineWeb rows and 2 biography rows of 64 tokens, for 120 steps. Each setting ran once.

Measures. Web loss is the validation loss on 256 FineWeb rows with the memory on. Recall is the share of test questions where the right answer has the highest probability among all answers of its kind; chance is about 4.5%.

Baselines. All are simplified re-implementations in one file, `bio_run.py`, so they share the model, data and optimizer.

- **Exact hints** (record #367): keys of 2, 3, 4 and 6 tokens, hints at three places.
- **Engram-style**: the last 2 and 3 tokens hashed into 50,021 buckets each, embeddings as wide as the model, a sigmoid gate from the dot product of the hidden state and the looked-up vector, added after the first block. No case compression.
- **CoCoNTs-style**: the longest exact hint joins the training target with weight equal to its share; no hints at test.
- **Memory layer**: product keys, 128 × 128 keys, 16,384 learned values, top 32 mixed by softmax, added after the first block.

- **Exact table mixed at test** (infini-gram style), on the model trained without memory.

4.2 The table alone

Before any model, we check how often the top hint at the answer position is the right answer. Exact keys of 2 to 6 tokens are right on 0% of test questions, because the test wording is never stored; on the stored wording the 6-token key is right on 98%. SetGram keys of 4 rare tokens are right on 96% of test questions and keys of 3 on 67% to 79%. So a reworded question can find its fact only through the set key.

4.3 Only SetGram recalls reworded facts

Table 2 and Figure 2 show the main result, with facts in the data twice. Every baseline stays between 3.0% and 6.7% on reworded questions, at chance. Exact hints and the exact table at test time recall the stored wording (.487 and .980), which shows that they work as intended and fail only because of the wording. SetGram hints in training reach .812 for people the model trained on and .795 for people only in the table. Mixing the SetGram table into the output of the model trained without memory reaches .963 and .958.

Memory	Web loss	Trained, new wording	Table only, new wording	Table only, stored wording
None	7.807	.067	.038	.032
Exact hints (record #367)	7.428	.057	.030	.487
Engram-style hashed 2- and 3-grams	7.808	.047	.040	.038
CoCoNTs-style targets	7.761	.060	.043	.047
Memory layer, product keys	7.806	.062	.055	.045
Exact table mixed at test	7.348	.067	.038	.980
SetGram hints	7.886	**.812**	**.795**	.757
Exact + SetGram hints	7.458	.478	.702	.845
SetGram table mixed at test	7.670	**.963**	**.958**	.958

Table 2: Facts in the data twice (`-once`). Width 32, 120 steps, one run each. Recall is the share of questions whose right answer is the top pick among answers of that kind (chance about .045).

The Engram-style memory and the memory layer fail for different reasons. At the answer position the Engram key is the last 2 or 3 tokens, "born in" or "was N born in" with the words in a different order than in training, so it either names no person or never occurred. The memory layer can store facts only through training; people 200 to 399 were never trained on, and for trained people 120 steps were not enough for the model or the memory layer to learn them.

4.4 Two tables in training can mislead each other

Exact and SetGram hints together recalled only .478 for trained people. On the reworded question, short exact keys such as "born in" fire with the answer of whichever person was most common after them, and the model, which learned to trust exact hints for web text, follows them. In the made-up fact language of Part II the same drop appears and grows with width (Table 4). A gate that learns which table to trust per position, as the taught switch of Section 12 does for exact hints, is the natural fix, which we did not test.

4.5 Facts stored once more: trust follows counts

With facts in the data three times (two training wordings plus the stored one), SetGram hints reached .923 for trained people but only .050 for people only in the table (Table 3). A table-only

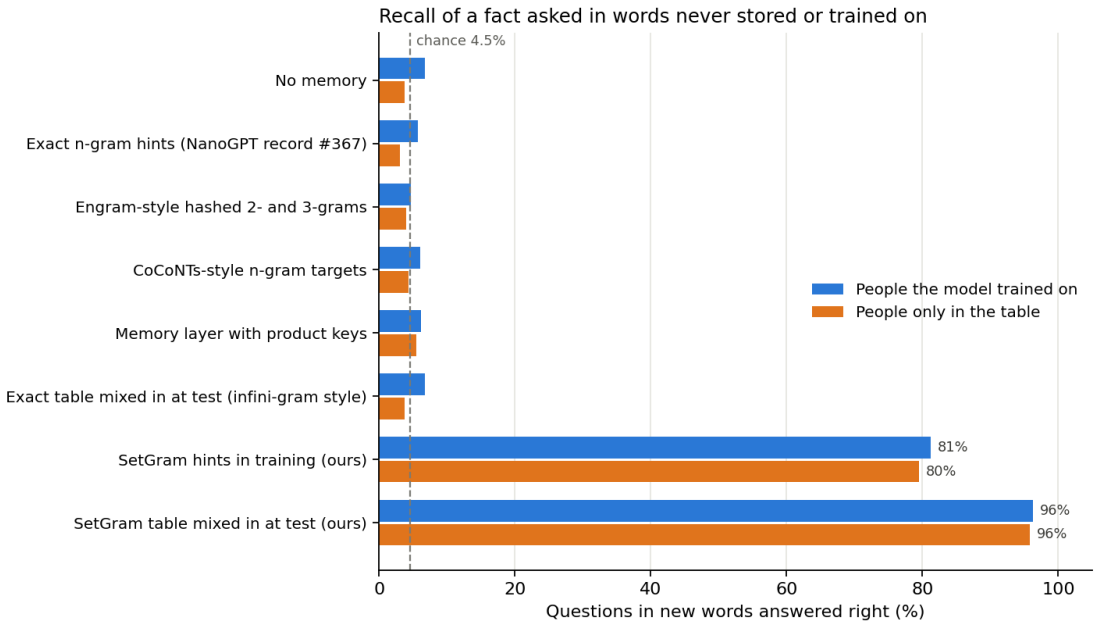


Figure 2: Recall of a fact asked in a wording never stored or trained on (Table 2).

person's fact has count 1 in the table, and on web text a count-1 hint is almost always wrong, so the trust for that kind stays low. With facts in the data twice, trained facts also yield count-1 hints that are right, and the model learns to trust them. Mixing at test time, where the trust is a single number λs, is not affected. In a real corpus, facts that matter appear with many counts, so this matters most for rare facts.

Memory	Web loss	Trained, new wording	Table only, new wording	Table only, stored wording
None	7.897	.047	.045	.045
Exact hints	7.483	.050	.033	.395
Engram-style	7.899	.052	.048	.038
CoCoNTs-style targets	7.844	.060	.052	.042
Exact table mixed at test	7.397	.047	.045	.980
SetGram hints	7.944	**.923**	.050	.057
Exact + SetGram hints	7.500	.898	.005	.287
SetGram table mixed at test	7.754	**.963**	**.958**	.958

Table 3: Facts of trained people in the data three times. Width 32, 120 steps, one run each.

4.6 The recall grows with training and width

For people only in the table, SetGram hints recalled .50, .67 and .80 at 30, 60 and 120 steps, and at 60 steps .67, .77 and .79 at widths 32, 64 and 128 (1.2M to 5.0M weights). Exact hints at width 128 stayed at .043. The width range is 4 times and the step range 4 times; longer runs did not fit our 10-second budget per run. Since an exact key cannot match a reworded question at any model size, we expect the gap to stay at larger scale, but we did not test it.

4.7 The same result in a made-up language

The made-up fact language of Section 10 has 1000 rare facts stored in the table in one wording and asked in another, never trained on. Content words are names, digits and relations. Table 4 shows SetGram from .737 to .853 over 16 times more steps and 4 times more width, while exact hints stay at or below .010. On the stored wording, SetGram scores .76 to .86 against 1.0 for exact keys: the SetGram table itself is right on 88%, and the rest are collisions where "x y z" and "z y x" share a key.

Hints	W32, 50 steps	W32, 200	W32, 800	W64, 200	W128, 200
None	.000	.009	.011	.009	.015
Exact	.009	.007	.005	.010	.008
SetGram	**.737**	**.840**	**.850**	**.852**	**.853**
Exact + SetGram	.684	.762	.778	.649	.634

Table 4: Rare facts asked in a second wording, made-up language, 2 layers, right of 1000, one run each.

4.8 On general web text SetGram adds little to exact keys

To see whether set keys also lower web loss, we built a variant that keeps the previous token exact and adds the set of the last 2 or 3 rare tokens, with a 25M-token table (Table 5). On top of the record's long keys of 6, 10 and 20 tokens, it lowered validation loss by .092 at 35 steps, shrinking to .047 at 140 steps and to .031 at width 96. On top of all exact lengths from 2 to 20 it added only .003 to .010: on general text, set keys mostly repeat what short exact keys already predict. Used alone in the biography runs, SetGram hints cost web loss (7.886 against 7.807 with no memory). SetGram is a memory for facts, to be added beside an exact table.

Steps, width	Exact 6,10,20	+ SetGram	Gain	Exact 2–20	+ SetGram	Gain
35, 32	8.523	8.431	.092	8.121	8.115	.007
70, 32	7.909	7.840	.069	7.524	7.513	.010
140, 32	7.743	7.697	.047	7.355	7.348	.006
70, 64	7.822	7.783	.040	7.435	7.432	.003
70, 96	7.765	7.734	.031	7.384	7.375	.009

Table 5: FineWeb validation loss with hints on, 2 layers, 25M-token table, one run each.

4.9 CoCoNTs reproduces in our toy

To check that our CoCoNTs-style baseline is not broken, we reproduced its web text gain. Training toward (1 − w) the true token + w the table's longest-match token, with w = $T\times$ share, lowered validation loss with no table at test by .057, .063 and .071 at 35, 70 and 140 steps (T = 0.5, width 32) and by .063, .069 and .063 at widths 32, 64 and 96. So the baseline improves web text as published, and its failure on reworded facts comes from its exact keys.

4.10 Word sets on a finished model

We also tried word-set lookups on MiniCPM5-2B, unchanged (Section 13), with a table of 1000 Wikipedia pages and 18 new made-up pages, on 18 reworded questions. Raising the scores of

candidate answers found by whole-word keys gave 4 right against 1 with nothing; a 20-word snippet from the best match gave 7, the same as putting the best passage in the prompt (81 extra tokens against 28). A model that never trained with the table cannot tell which candidate answers the question, which supports training with SetGram hints rather than only adding them at inference to a model that never saw them.

Part II
Exact N-gram Hints in Training

5 Leakage, Memorizing and Reasoning with Exact Hints

Brunborg [1] builds the record's hint corpus with idle CPUs during training. The model receives the most common next token as a hint and learns how much to trust it. The blog post reports two things that matter here. A large hint corpus from the start makes the backbone rely on the hints too much, so training uses a small corpus and validation a large one. And each doubling of the corpus cuts the loss more than the last [1].

The record's code computes training hints in a strictly causal way: a step's hints count only tokens trained in earlier steps [3]. That has a cost. At validation the model meets hints from a far larger corpus than any it trained with, so it must trust counts it has never seen.

We ask whether a language model trained with Muon can use hints from the whole corpus from the first step, without the failure the blog post describes, and what hints do to memorizing and reasoning. Removing a token's own occurrence fixes the failure in validation loss, so by that measure a full table is safe. It does not protect the model's own learning: with the occurrence removed, a full table in training still stops the model from memorizing and from learning comparison, and a small table in training, the record's choice, keeps comparison. The same leave-one-out, trust buckets and hint places are used by SetGram in Part I.

6 Background

Muon. Muon [4] is an optimizer for the matrices of a network. It keeps momentum, replaces the update with its nearest orthogonal matrix using Newton-Schulz steps, and moves the weights by a fixed step [5]. Wang et al. [6] find that Muon learns rare classes better than Adam, the same rare-data problem hints address. Embeddings stay on Adam.

Memory by lookup. Section 2 surveys memory methods. Hints differ from Engram-style tables in that nothing is learned per n-gram, and the corpus stores only the next tokens.

Leakage. A count feature that includes the row's own label leaks the label, as in target encoding [11]. CatBoost [12] fixes this with ordered statistics, which use only earlier rows, the same idea as causal hints. Leave-one-out statistics are the other common fix, and RETRO filters out neighbours from the same document [10]. Our searches found no paper comparing causal and leave-one-out hints for n-gram hints, and none using hint dropout, which follows dropout [13].

7 Method

Model. We use the Muon lab model [14]: a character-level transformer on Tiny Shakespeare with RoPE, QK-norm, RMSNorm, a SwiGLU MLP, no biases and tied embeddings, with 4 layers and 4 heads. The first 90% of the text is the training split. Muon trains the hidden matrices with momentum 0.95, and Adam trains the embeddings and norm weights at 0.003. Training has 20 warmup steps and a linear cooldown over the last 30%.

Hints. We follow the blog post, with lengths 6, 12 and 24 characters in place of 6, 10 and 20 tokens. For each length the hint is the most common next character. Its count falls into one of 7 buckets and its purity (the share of the n-gram's occurrences followed by that character) into one of 6, giving 126 kinds. Each kind has a learned amount σ and a learned vector β. The hint vector is

$$H_t = \sum_n \left(\sigma_{k(n,t)}\, E[\text{hint}_n(t)] + \beta_{k(n,t)}\right), \tag{3}$$

where E is the embedding table. We add H_t after the embedding, after layer 2 and before the output head, each with its own learned scale. The hint parameters use Adam at 0.03.

Training hint policies. Validation always uses the whole training split as its corpus. Training uses one of these:

- **None:** no hints.
- **Own token kept:** the whole training split, with each token's own occurrence still in the counts.
- **Earlier steps:** only tokens trained in earlier steps, as in the record.
- **Leave-one-out:** the whole training split with each token's own occurrence removed. If removing it leaves no count, there is no hint, exactly as for an unseen n-gram at validation.
- **Leave-one-out with dropout:** the same, and each hint is dropped with probability 0.5.

Training streams shuffled, non-overlapping chunks of 65 characters, so a token appears at most once per pass. The default batch is 8 and the default length is 200 steps, or 102K trained tokens. The corpus is 1.0M characters, 10 times the tokens trained. The record's ratio is about 520 times [1], which a 1 MB text cannot reach.

Measurement. Each setting was run once at Muon learning rates 0.01, 0.02 and 0.04, and we report the best, on 96 fixed validation chunks.

8 Results

8.1 Keeping the token's own occurrence breaks training

Table 6 shows the results at width 64. With the token's own occurrence in the counts, every training n-gram has a match, and a long n-gram usually has only that one occurrence, so the hint is the answer. The model learns to trust hints it will not get at validation, and its loss with hints is 3.53 against 2.16 with no hints at all. Removing that one occurrence brings the loss to 1.84.

Training hints	Validation, hints on	Validation, hints off
None	–	2.165
Own token kept	3.525	4.735
Earlier steps	1.890	2.307
Earlier steps, half dropped	1.860	2.204
Leave-one-out	1.843	2.508
Leave-one-out, quarter dropped	1.800	2.349
Leave-one-out, half dropped	**1.805**	**2.256**
Leave-one-out, three quarters dropped	1.807	2.206

Table 6: Width 64, 200 steps, batch 8. Best of three learning rates by loss with hints.

8.2 Hint dropout reduces the loss penalty with hints off

Leave-one-out alone makes the backbone worse when hints are off: 2.508 against 2.165 with no hints, which matches the blog post's report of relying on hints too much. Dropping half of the hints brings this to 2.256 and lowers the loss with hints from 1.843 to 1.805. Dropout also helps earlier-steps hints (1.890 to 1.860), but they stay behind. Dropping a quarter, half or three quarters gives losses within 0.007 of each other, so half is a default and not a tuned best. This measures overall loss only. On memorizing, dropout does far less: Table 13 shows that a model needs hints removed from 90% of its tokens to match a model that never saw hints.

8.3 The gain stays the same as the model grows

Table 7 shows a gap between 0.070 and 0.085 across a 15-fold range of parameters.

Width	Parameters	No hints	Earlier steps	Leave-one-out, half dropped	Gap
32	56K	2.317	1.938	1.858	0.081
64	209K	2.165	1.890	1.805	0.085
128	812K	2.085	1.840	1.769	0.070

Table 7: Validation loss with hints on, 200 steps, batch 8, corpus 10 times the tokens trained.

8.4 The gain depends on corpus size divided by tokens trained

Table 8 changes the tokens trained at width 32 with the corpus fixed. The gap is 0.370 at a ratio of 40, 0.081 at 10 and 0 at 2.5, because hints from earlier steps approach the whole corpus as training goes on. At 3200 steps a second pass starts, so hints from earlier steps contain each token's own earlier occurrence and the failure above returns: loss rises to 2.533, while leave-one-out reaches 1.603. Causal hints are safe only within one pass.

8.5 Each doubling of the corpus

With random 256-character pieces of the corpus, the gains per doubling are 0.040, 0.052, 0.050, 0.060, 0.099, 0.062 and 0.061 for leave-one-out with dropout, and 0.053, 0.055, 0.052, 0.060, 0.073, 0.028 and 0.018 for earlier steps (Figure 3). Neither curve shows gains that rise with each doubling, so we did not reproduce the blog post's result on Shakespeare. In the earlier-steps model the loss on

Steps	Batch	Ratio	No hints	Earlier steps	Leave-one-out, half dropped	Gap
200	2	40	2.534	2.386	2.016	0.370
200	8	10	2.317	1.938	1.858	0.081
800	8	2.5	1.956	1.676	1.676	0.000
3200	8	0.6	1.755	2.533	1.603	0.930

Table 8: Width 32, hints on. Ratio is corpus size divided by tokens trained. The last row is past one pass over the data.

hinted characters stops falling after 251K characters, about 2.5 times its tokens trained, because it never learned how much to trust larger counts. In the leave-one-out model it keeps falling to 1.66.

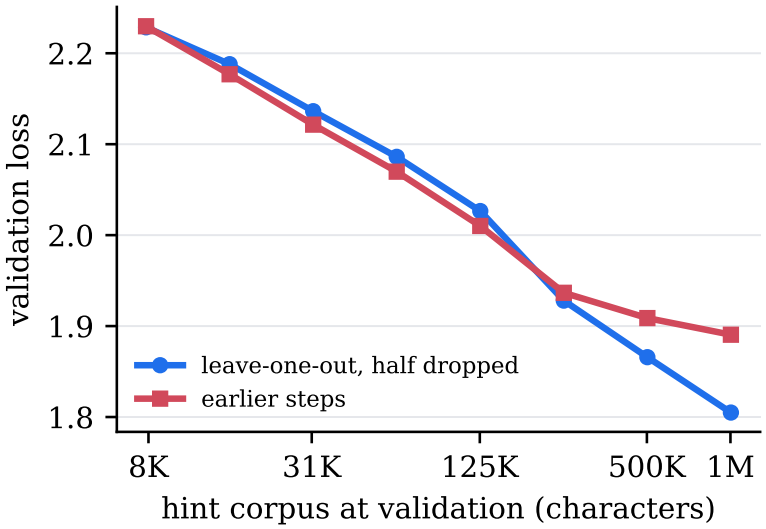


Figure 3: One final model of each kind evaluated with random subsets of the corpus. Width 64, 200 steps.

9 Web Text

We repeated the test on FineWeb [1], using GPT-2 tokens from one 100M-token training shard as the corpus and text from the separate validation shard, with corpus sizes from 0.78M to 100M tokens. The model has width 40, 4 layers, about 1.4M parameters, batch 8 and 90 steps (46K trained tokens) unless stated. Hints use lengths 6, 10 and 20 tokens unless stated; the longest set adds 2, 3, 4 and 5.

9.1 The hints improve with each doubling on web text

Before any model is involved, the share of validation tokens where the hint is right grows faster with each doubling: each doubling adds 0.07% of tokens at 1.56M and 0.71% at 100M, where the hint is right on 2.06% of tokens. This matches the blog post's report for the hints, unlike our Shakespeare result. We did not check the cause; near-duplicate pages in both shards are one possibility.

9.2 Leaving out the whole page removes the harm on small corpora

An earlier sweep with 256 validation chunks compared hints from earlier steps with leave-one-out and half dropped: 7.860 against 7.835 at 100M, but leave-one-out lost below 25M (7.884 against 7.889 at 0.78M), and the 46K-token stream found 0.36% of tokens with a hint or fewer, so earlier-steps hints barely trained. Validation loss without hints was 7.871. Removing only a token's own occurrence does harm below 25M tokens: the overall loss is 7.877 at 0.78M. Removing every match from the token's own page removes this harm. At 100M the loss is 7.816 with the page left out and no hints dropped, against 7.821 with one occurrence left out and half of the hints dropped, and with the page left out dropping half no longer helps (7.825).

9.3 Short matches give most of the gain, and that gain shrinks with training

Adding matches of 2, 3, 4 and 5 tokens raises the share of tokens with a hint from 5% to 92% and gives by far the largest drop, from 7.816 to 7.244 at 100M. Most of this comes from lengths 2 and 3: length 2 alone gives 7.373, and length 4 and longer gives 7.696.

Table 9 shows longer training at corpus 100M. The gain from short matches falls from 0.63 to 0.34 over 16 times more training, because the model learns common next tokens by itself. The gain from lengths 6, 10 and 20 stays between 0.04 and 0.07. The model with all seven lengths depends on them: with hints off it scores 7.754 against 6.916 at the longest run. So this gain is recall during validation and not faster learning, since the model's own predictions are worse than those of a model trained with no hints.

Steps	Tokens trained	No hints	Lengths 6, 10, 20	All seven lengths
90	46K	7.871	7.816	7.244
360	184K	7.398	7.333	6.879
1440	737K	6.916	6.872	6.580

Table 9: Validation loss at corpus 100M. Hints leave out the token's own page. The vocabulary grows with the steps (28,359, 39,394 and 46,243 tokens), so compare within a row.

10 Facts and Reasoning

Engram [7] reports gains on reasoning benchmarks as well as knowledge benchmarks, with learned tables, and RETRO's gains largely come from tokens shared with the test data [22]. We found no test of retrieved next-token hints by task kind, so we built a language where the kind of each answer is known.

Setup. The corpus holds 1.5M lines of made-up text: rare facts stated 3 times each, common facts stated 300 times, sums modulo 10 with 20 of 100 pairs held out, two-step comparisons with fresh names, and reading an answer from the line. Hints use lengths 2, 3, 4 and 6, leave out each token's own occurrence, and enter after the embedding, after layer 2 and before the output head. The model has width 64 and 4 layers, and trains for 5760 steps of 32 lines at learning rate 0.02. We test 1000 new lines per group. Chance is 1% for facts, about 10% for sums and 50% for reading and comparison, where always picking the first name scores that much.

Hints	Common	Rare	Other words	Not in table	Held-out sum	Read	Comparison
None	.995	.009	.005	.013	.492	.480	1.000
All 3 places	1.000	1.000	.010	.004	0	.502	.495
All 3 places, half dropped	1.000	1.000	.006	.012	0	.513	.495
Input only	1.000	1.000	.006	.012	0	.513	.497
Length 6 only, all 3 places	1.000	1.000	.012	.010	.730	.474	.499
Output only	1.000	1.000	.010	.003	0	.924	1.000
Output only, half dropped	**1.000**	**1.000**	**.006**	**.012**	**.536**	**.499**	**.999**

Table 10: Accuracy at the answer token.

Results. Table 10 shows that hints take rare facts from chance to 100% when the test uses the stored words, and give nothing when the words change or the fact is absent. Hints at all three places stop the model from learning comparison (.495 against 1.000) and held-out sums (0 against .492). Hint dropout alone does not fix this. Hints only at the output restore comparison, and with half dropped they also restore sums and keep all recall. Section 12 tests why.

This does not contradict Engram: its tables are learned, while our hints are retrieved next tokens. A made-up language shows a mechanism, not its size in real models.

10.1 Web text and reasoning in one model

Next we trained one model on both languages at once, with 2 web chunks and 32 made-up lines per step for 5760 steps. The two languages share no tokens, so one joined hint table gives the same hints as two tables. Web hints use all seven lengths and leave out the token's page, with corpus 100M.

Width	Hints	Web, hints on	Web, hints off	Held-out sum	Comparison
32	None	–	7.132	.493	.600
32	All 3 places	6.824	7.846	0	.496
32	Output only, half dropped	7.095	7.107	.458	.929
64	None	–	6.991	.350	.971
64	All 3 places	6.714	7.637	.048	.496
64	Output only	6.878	7.061	0	1.000
64	Output only, half dropped	6.971	6.991	.267	.995
96	None	–	6.956	.484	.981
96	All 3 places	6.677	7.602	0	.495
96	Output only	6.777	6.988	0	.995
96	Output only, half dropped	6.863	6.942	.434	.996

Table 11: One model trained on web text and the made-up language. Accuracy at the answer token. Rare facts in the table scored 1.000 in every hint row.

Table 11 shows a trade-off. Hints at all three places give the largest web text gain, 0.28 to 0.31, and block comparison at all three sizes; with hints off their web loss is 0.65 to 0.71 worse than no hints. Output-only hints keep comparison and keep 41% of that gain at width 64 and 64% at width 96. Dropping half restores held-out sums too but keeps only 12%, 7% and 33% of the gain at widths 32, 64 and 96.

11 How Rare Text Must Be for Hints to Help

Recall in language models tracks how often a fact appears [15, 16], and one study reports verbatim recall near 64 exposures at 1B parameters and 16 at 8B [17], while another finds the effect disappears on some models [18]. For retrieval, kNN-LM papers disagree on whether rare patterns [19] or high-frequency tokens [20] gain most. We found no such split for retrieved next-token hints.

11.1 A model needs only a few statements of a fact

Each of 200 facts per group was stated exactly 1, 2, 4 up to 256 times in training, with nothing else in the data. The table holds every fact, so a hint is always right. Table 12 shows accuracy without hints. The model reaches 50% at about 45, 19, 10, 5, 4, 3, 3 and 2 statements at widths 16, 24, 32, 48, 64, 96, 128 and 192, so a wider model needs fewer statements. This agrees in direction with [17] and with a width bound for memorizing facts [27]. Going from 2 to 8 layers did not move it, so the count does not follow the number of weights. With 4 times more training the same count gives the same accuracy, so the count matters and not the length of the run. With hints every group scores 100%, but a model trained with hints scores at most 2.5% with hints off, even at 256 statements: it memorized almost nothing itself.

Statements in training	1	2	4	8	16	32
Width 16	.000	.035	.005	.065	.105	.260
Width 24	.020	.025	.075	.175	.395	.885
Width 32	.030	.045	.130	.380	.820	.995
Width 48	.055	.130	.350	.775	1.000	1.000
Width 64	.060	.165	.535	.870	.990	1.000
Width 96	.120	.275	.670	.935	1.000	1.000
Width 128	.125	.310	.735	.970	1.000	1.000
Width 192	.185	.435	.775	.965	.995	1.000
Width 64, 2 layers	.070	.175	.470	.850	.995	1.000
Width 64, 8 layers	.075	.175	.530	.865	1.000	1.000
Width 64, 400 facts per count	.068	.145	.360	.770	.965	1.000
Width 64, 4 times more training	–	–	.485	.885	.995	1.000

Table 12: Accuracy without hints by how many times a fact was stated. Chance is 1%.

11.2 Keeping a model memorizing while it uses hints

A model that always gets hints stops memorizing, so we tried ways to keep both, at width 64 (Table 13). Taking the hints away from a random share of tokens works only at a high share: 25% still leaves 70% accuracy at 256 statements, and 90% matches the model that never saw hints. Two methods keep 100% with hints and full memorizing. Training every step on the same batch with and without hints matches the model that never saw hints at every count, for 28% more clock time. Keeping only text seen under 16 times in the table gives 100% without hints at 16 statements and above, where the model gets no hint. The closest work we found drops privileged inputs in reinforcement learning [28], and R-Drop also runs two passes per batch, on the same input [29].

Statements in training	1	4	8	16	32	64	256
No hints in training	.060	.535	.870	.990	1.000	1.000	1.000
Hints always	.000	.000	.000	.000	.000	.005	.005
25% of tokens without hints	.005	.010	.015	.005	.010	.140	.700
50% of tokens without hints	.020	.010	.030	.060	.285	.915	1.000
75% of tokens without hints	.045	.235	.555	.960	1.000	1.000	1.000
90% of tokens without hints	.070	.425	.785	.985	1.000	1.000	1.000
No hints in last half	.045	.225	.515	.865	.995	1.000	1.000
No hints in last quarter	.005	.040	.055	.175	.375	.765	1.000
Hints at output only	.020	.000	.010	.000	.010	.030	.010
Every step with and without hints	.065	.475	.890	.985	1.000	1.000	1.000
Table keeps text seen under 16 times	.015	.025	.000	1.000	1.000	1.000	1.000

Table 13: Accuracy with hints off, width 64, one run each. With hints on, every row scores at least 99% except no hints in the last quarter.

11.3 Only training with and without hints survives web text and reasoning

We repeated the best methods in the joint model at width 64 (Table 14). Training every step with and without hints keeps 96% of the web text gain with hints on, and with hints off it beats the model that never saw hints by 0.039 and recovers sums and comparison, for 47% more clock time. With hints on, reasoning stays blocked, and a common fact reaches only 7% with hints off against 78% without hints. A table of only rare text loses the web text gain and comparison stays at chance.

Training	Web on	Web off	Sum on/off	Comparison on/off	Fact off	Gain
No hints	–	6.991	.350	.971	.780	–
All three places	6.714	7.637	.048/.000	.496/.014	.012	100%
With and without hints	**6.726**	**6.952**	.040/.403	.500/.960	.070	96%
Output only, with and without	6.924	6.980	.270/.270	.979/.979	.036	24%
75% of tokens without hints	6.868	6.991	.222/.222	.914/.774	.270	44%
Table under 16 times	7.010	7.077	.581/.581	.495/.505	1.000	0%

Table 14: One model on web text and the made-up language, width 64, 5760 steps, one run each. Hints enter at all three places unless named. Gain is the share of the 0.277 web text gain of hints at all three places. Fact off is a common fact with hints off.

11.4 On web text the gain sits on text the model itself has not seen

We split the gain of 0.357 at 1440 steps over 65K validation tokens (Table 15). Gain per token rises with match length, from 0.22 at length 2 to 3.98 at length 20, but lengths 2 to 4 hold 72% of the gain because they cover far more tokens. Hints that are right, on 18% of tokens, give 125% of the gain, and wrong hints take back 20%.

The model trained on only 737K tokens, 1/135 of the table, so text seen 16 times in the table is usually text the model never saw. Table 16 counts what the model itself saw: 79% of the gain sits on contexts it never saw, and gain per token falls from 0.42 at zero sightings to about zero at 64 or more, which agrees with the facts test. These models sit at a loss near 6.9, so this describes early training only.

Group	Share of tokens	Share of gain	Gain per token
No match	8%	−4%	−0.17
Longest match 2 to 4	80%	72%	0.32
Longest match 5	7%	11%	0.56
Longest match 6	4%	12%	1.05
Longest match 10	0.6%	5%	3.16
Longest match 20	0.3%	3%	3.98
Match seen once	19%	28%	0.53
Match seen 2 to 15 times	35%	39%	0.40
Match seen 16 or more times	38%	37%	0.35

Table 15: Web text gain of hints of all seven lengths. The two blocks split the same tokens.

Times the model saw the context	Share of tokens	Share of gain	Gain per token
0	67.2%	79.2%	0.421
1	7.9%	8.3%	0.377
2 to 3	4.7%	5.1%	0.393
4 to 15	5.0%	3.7%	0.265
16 to 63	2.3%	0.9%	0.138
64 or more	1.5%	−0.1%	−0.026

Table 16: Web text gain split by sightings in the model's own 737K training tokens, for the 88.6% of tokens whose longest match fits inside their chunk.

12 How Hints Block Reasoning

12.1 Training with hints from the start blocks the skill more than adding hints late

To find the cause we built a test that runs in seconds: 2 layers and 1000 steps on half facts and half comparison lines "a > b . b > c . top ? a .", with the two statements in random order and name triples held out of training. Hints go in at all three places. At width 32 the shared table gives comparison .545, and the model says the first name in the line 94% of the time, which is right half the time.

We trained a model with no hints for 600 steps, then trained only the hint trust for 200 steps with the model frozen, then trained everything with hints on for 800 more steps (Table 17). Hints at full strength took facts from about .06 to .80. Comparison stayed level at width 32 and at 4 layers (.946 to .958, .931 to .950) and fell at widths 64 and 128 (.974 to .853, .996 to .809). Further training with hints then removed the skill at width 32 and at 4 layers (.504, .580) and left it about the same at widths 64 and 128 (.852, .872). A model trained with hints from the start was lowest or level with the lowest at every size (.505, .691, .522, .499). So the block comes more from training with hints than from the hints alone, and adding hints late saves the skill only at the two larger widths. Hints with the shared table already solve training lines: after 600 steps of always-on hints the model scores .999 on them and .52 on held-out names.

Hand settings of hints point the same way but do not settle the cause. In runs trained with and without hints, comparison is .764 with hints everywhere, .996 with hints only at the answer word and .516 with hints everywhere else. Turning off one hint, at the dot between the statements, gives 1.000 at width 32 but .553 at width 128.

Comparison	Width 32	Width 64	Width 128	4 layers
Trained with no hints, hints off	.946	.974	.996	.931
Then hints on, only trust trained	.958	.853	.809	.950
Then 800 more steps with hints on	.504	.852	.872	.580
Hints on from the start	.505	.691	.522	.499

Table 17: Comparison on held-out names with hints on unless stated, one run each.

12.2 A small table in training keeps reasoning and improves facts

Brunborg trains with a small corpus and validates with a large one, because a large corpus from the start makes the model rely on hints too much [1]. We tested that choice on reasoning (Table 18). The training table reads a share of the lines the model does not train on, the test table reads all of them, and test counts are scaled to the same count levels so that trust learned in training applies. Hints stay on, with no switch and no second pass. The model trains on 16,000 comparison lines and the full table holds 84,000 others, so 5% to 60% is 0.25 to 3 times the lines trained on. In this range comparison is .97 or above at all three widths and facts reach about .88, above the shared table's .81. At 2% comparison falls at width 32. From 70% to 95% the result is erratic, and at the largest table comparison is back at chance at every width. We cannot explain the break. Training lines score near 1.0 in the good and the bad runs, and from 20% up the hint at the answer word is right equally often on training lines at every size (.50 for the longest hint, .55 for any length), so the hint at the answer does not explain it.

A separate table also works. With the table reading one half of the name triples and training using the other half, comparison is 1.000 at widths 32, 64, 128 and at 4 layers, and at 4000 steps at width 32, with facts at .80 to .82. With hints off comparison stays low (0 to .23) in these runs, so the model needs its hints.

Share of table	Times the lines trained on	Width 32	Width 64	Width 128	Facts
2%	0.1	.781	–	.957	.81–.87
5%	0.25	**.996**	**.996**	**.994**	.88–.89
10%	0.5	**1.000**	–	**1.000**	.88–.89
20%	1	**1.000**	**1.000**	**1.000**	.87–.88
30%	1.6	**1.000**	**1.000**	**1.000**	.87
40%	2.1	**.997**	**.990**	**.990**	.87
50%	2.6	**.999**	–	**1.000**	.88
60%	3.2	**.972**	**.984**	**1.000**	.89
70%	3.7	.897	.945	.785	.89
75%	3.9	.549	–	1.000	.89
80%	4.2	1.000	.877	1.000	.88–.89
90%	4.7	.986	.976	.823	.88–.89
95%	5.0	.982	.504	.541	.89
100%	5.3	.523	.597	.503	.89

Table 18: Comparison with hints always on, 1000 steps, one run each, all testing with the full table. Bold marks the range where every width is at .97 or above. We did not run this on web text or on sums.

12.3 The limit follows copies in the table, not table size

The break could depend on table size against the data trained on, or on how many copies of each line the table holds. We kept the same 84,000 table lines and 16,000 training lines and changed the number of names, which changes copies of each line and nothing else (Table 19). With 30 or 40 names the full table no longer blocks comparison, and facts rise to .97 and .99. Comparison stays high at few copies and falls at many; the next subsection maps the limit. Changing training steps fits copies better than the ratio: 2000 steps is 2.1 times the lines trained on, inside the safe ratio, but 5.8 copies, and gives the erratic .920 and .720.

Several explanations failed. Length 6 hints are the only ones that hold all three names, but without them the full table still blocks comparison (.504, .508 and .448 at widths 32, 64 and 128), and length 4 hints alone block it too (.492 and .501). Giving every hint one trust level, with no count or share signal, leaves the 100% table blocked (.590, .529) and the 20% table working (.982, .978), with facts at .89, so the count scaling above is not needed here. A 20% table used in training and testing keeps comparison at 1.000 with facts at .66 and .75, so the block comes from the table in training and the full table at test adds only facts. In a blocked model, removing the hint at the word "top" raises held-out comparison from .52 to .80 at width 32 and from .50 to .61 at width 128. We map the limit and look for the cause next.

Setting	Copies of each line	Width 32	Width 128
14 names, full table	35	.552	.560
20 names, 500 steps	7.9	.552	.884
20 names, full table	7.2	.523	.503
20 names, 90% of table	6.5	.986	.823
20 names, 2000 steps	5.8	.920	.720
20 names, 60% of table	4.3	**.972**	**1.000**
30 names, full table	1.8	**.999**	**1.000**
40 names, full table	0.7	**1.000**	**1.000**

Table 19: Comparison with hints always on, one run each. The 14, 20, 30 and 40 name full tables hold the same number of lines. A copy here is one more table line with the same three names in the same order and the same statement order, so each triple has two versions.

12.4 Copies of the training sentences block the skill at every name count

Table 20 sets copies directly at four name counts. Up to 5 copies comparison was .94 or above in all 32 runs, 7 and 10 copies gave mixed scores, and at 15 or more copies 11 of 14 runs stayed near .50. Copies of the same sentence fit better than table size: the 40 name table at 15 copies holds 1.75 million comparison lines and the 14 name table at 30 copies holds 71,000 lines, and both block. These runs scale test counts by table lines over all lines (`-countscale 1`). Without scaling, the comparison score at 14 names falls to between .51 and .72 for 1 to 7 copies, and facts are .73 and .89 at 14 and 20 names, against about .1 to .2 at few copies with scaling, so the low copy cells at 14 names depend on how counts are matched. The limit of 5 copies therefore holds only with test counts scaled. Scaling has its own cost: in 12 of the 32 runs at 5 copies or fewer (every 14 name run, and 20 names at 1 and 2 copies) facts with hints were only .08 to .22, so those models mostly ignored their hints and those cells do not show hints and comparison working together. In the other 20 runs facts were .88 or above.

Only copies of the sentences being trained on matter. When the table reads one half of the name triples and training uses the other half, comparison is 1.000 at both widths at 17 and at 60

Names	1	2	4	5	7	10	15	30
14	.95/.95	.99/.99	1.0/1.0	.99/.99	.96/.98	.56/.53	.98/.50	.54/.57
20	1.0/1.0	1.0/1.0	1.0/1.0	1.0/1.0	.81/.98	.51/.71	.69/.52	.65/.48
30	1.0/1.0	1.0/1.0	1.0/.99	.99/.99	.99/.93	1.0/.54	.51/.49	.48/.51
40	1.0/1.0	1.0/1.0	1.0/1.0	1.0/1.0	.99/1.0	.84/.49	.48/.48	–

Table 20: Comparison score at width 32 / width 128 by copies of each line in the table, with hints always on, one run each. A dash marks runs we skipped because the table is too large.

copies of each table line, with facts at .80. Repeating training lines does not block either: 1000 lines trained 16 times each, with 2 copies in the table, score .999 and 1.000. Blocked models also read the table: with 30 copies of the training lines, adding the held-out test sentences to the test table 30 times raises their score from .65 and .48 to 1.000 at both widths, while adding them 7 times gives .67 and .48. So the blocked model has not learned comparison. It depends on the hints a sentence gets when the table holds it about as often as the training sentences.

The blocked model uses a simple shortcut. On held-out lines it says the first name in the line, which is right when the first statement comes first and wrong otherwise, so it scored 1.00 and about .00 on the two kinds of line, and .50 overall. On its training lines it scores 1.00 on both.

Removing hints at single word positions on comparison lines, in training and testing, shows where the block starts (20 names, 30 copies, widths 32 and 128). Without the hint at the first dot, the word between the two statements, comparison rises from .65 and .48 to .95 and 1.000, and across 14, 30 and 40 names it stayed above .90 in 5 of 6 runs. Removing the hint at any of the other single words left at least one width blocked; removing the word 7 hint gave .92 and .48. With only the answer hint kept, comparison reaches .996 and .999, so the answer hint alone does not block. Removing the answer hint does not fix the block (.48 and .65), so the block does not come from copying the answer hint. We do not know why the hint at the first dot matters.

The block held as models grew. At 30 names, 2 copies gave 1.000 at widths 32, 64, 128 and 256, at 4 layers and at 4000 steps. At 30 copies comparison stayed at .48 to .51 at those four widths and at .50 at 4000 steps at width 128, but it escaped twice: .995 at 4 layers and .973 at 4000 steps at width 32. Each result is one run, on a made-up language.

12.5 A switch learns to shut hints only when taught

We multiply each token's hint by a switch between 0 and 1, read by a linear layer from the hidden state of a pass without hints. Learned from the loss alone, the switch stays at 1.0 and comparison stays at .769. We then gave it a second target: predict whether the longest hint equals the next token. This target is known from the data and adds one small loss. In the small test the taught switch gives comparison between .50 and .996, and no place in the model does best at every size.

12.6 The taught switch also lowers web text loss

We added the taught switch to the joint model, at all three places, training every step with and without hints (Table 21). Comparison with hints on rises from chance to .905, 1.000 and 1.000. Web text loss with hints falls below plain hints by 0.043, 0.070 and 0.094 at widths 32, 64 and 96, so the switch does not trade the web text gain for reasoning. At width 64 this is 125% of the plain gain, and training with and without hints alone reaches 96%, so the extra gain comes from the switch. A common fact with hints off reaches .621 and .957 at widths 64 and 96 against .012 and .011 with plain hints. The switch averages .24 to .27 on web text hints. Held-out sums with hints on stay

blocked (0, .251, .094). With the switch at the input only, width 64 gives 6.671 on web text but comparison stays at .495, the reverse of the small test.

The lead depends on training length. At width 64 the switch is behind at 360 steps (7.155 against 6.921), level at 1440 (6.819 against 6.816) and ahead at 5760 (6.644 against 6.714), and comparison is not yet learned below 5760 steps. At 1440 steps a switch with no training pass without hints gives no gain and comparison .007; we did not run that arm at 5760 steps. The widths span only 3.2 times the parameters, each setting ran once, and the switch needs a pass without hints at every step, so it costs clock time we did not measure under equal load.

The closest work we found has no such target. Engram's gate reads the hidden state and the memory [7] and we found no separate target for it in the paper. kNN-Adapter learns a per-token mix on top of a frozen pretrained model [24], and SpecDec++ trains an acceptance head for speculative decoding at inference [26]. We found no gate trained during pretraining on whether the retrieved token is right.

Width	Web, no hints	Web, plain hints	Web, switch	Web, switch off	Comparison, plain/switch	Fact off
32	7.132	6.824	6.781	7.082	.496/.905	.014
64	6.991	6.714	**6.644**	6.952	.496/1.000	.621
96	6.956	6.677	**6.582**	6.918	.495/1.000	.957

Table 21: One model on web text and the made-up language, 5760 steps, one run each. Fact off is a common fact with hints off in the switch model.

13 Hints on a Finished Model

All earlier sections train the model with hints. Here we leave a finished model unchanged and only raise the output score of each hinted token, so training cannot remove a skill. The model is MiniCPM5-2B (2.5 billion parameters, 42 layers, Apache 2.0), in its 4-bit MLX form on a laptop. The closest earlier work is kNN-Adapter [24], which mixes a finished model's output with retrieval from a target-domain store.

The table stores the most common next token after word groups of 2, 3, 4 and 6 tokens. A learned trust number sets how far each hint raises the score. It depends on the hint length, how often the group was seen, how often that token followed it, and how likely the model already thinks that token is. Trust has 840 numbers and trains in about a second from about 10,000 places. Except for the row that tests pages stored in the table, table, trust and test text never share a file or page. The model reads 128 tokens at a time and each setting ran once on about 8,000 test places unless stated. All code is in `finished/`.

13.1 Hints recall stored text but miss reworded questions

The table read 70% of about 52,000 Wikipedia pages from an April 2026 copy [25], plus 48 pages about 2026 events; 27.6M tokens and 24.8M groups of 6 tokens. Trust trained on 15% of the pages and the test used the rest (18,840 places).

The row for 2026 pages tests lookup, not prediction: those pages are in the table, so most long matches return the page's own next word. The 11 facts were written by hand from pages in the table, once in the page's own words and once in other words. A reworded question misses the stored word group or meets a wrong hint: "the operating system on the Acer Googlebook 14 is" went from "Windows 10" to "a Linux-based", and the page says Google. For new facts asked in new words, search over documents remains the better tool.

Test	Hints off	Hints on
Unseen Wikipedia text, next word right	41.7%	43.1%
Unseen Wikipedia text, loss	3.145	3.086
2026 pages stored in the table, next word right	41.6%	83.0%
11 new facts, the page's own words	1 right	6 right
The same 11 facts, reworded	1 right	1 right
100 comparison questions (three names)	54 right	54 right
100 two-digit sums	1 right	1 right

Table 22: MiniCPM5-2B with and without hints. No comparison answer changed. The 4-bit model given a bare sum fails with or without hints, so the sums say nothing about harm.

13.2 The gain rises with exact repeats and with how low the model scores alone

We repeated the test on other text, with a new table and trust each time. Table 23 is sorted by the share of places where a 6-token hint exists. The share does not set the order of the gains by itself: pandas at 38% gains 1.9 points and the notes at 20% gain 11.

Text, table and test from different files	Table tokens	Off	On	6-token hint
transformers code, unseen model folders	6.3M	66.2%	**83.9%**	81%
transformers code	5.3M	65.7%	**79.2%**	69%
pandas code	3.8M	67.0%	68.9%	38%
sympy code	6.0M	64.4%	67.9%	35%
sklearn code	2.4M	64.8%	68.0%	34%
networkx code	1.3M	65.2%	68.7%	33%
scipy code	4.3M	61.3%	63.0%	24%
Own Markdown notes, exact copies removed	5.6M	32.1%	**43.1%**	20%
sympy code, 10% of table	0.45M	64.4%	65.1%	13%
Wikipedia, full table	27.6M	41.7%	43.1%	10%
Wikipedia, 10% of table	2.6M	42.0%	42.6%	4%
Table from transformers, test on torch	7.3M	58.6%	58.4%	4%
sympy code, 1% of table	0.04M	64.4%	64.4%	3%
Wikipedia, 1% of table	0.26M	42.0%	42.1%	1%

Table 23: Next word right with hints off and on, by kind of text.

Three things follow. A table from a different source gives nothing: transformers code did not help torch code. Holding out whole model folders did not lower the transformers gain, likely because model files in that library copy each other's code; transformers is the extreme case, and five other libraries gain 1.7 to 3.5 points. The notes gain 11 points at only 20% because the model starts low there, at 32.1%, on private text it has never read. With exact duplicate files left in, 27.6% of the notes by size, that row read 33.7% to 56.9%, so we removed them; near copies remain.

The share of 6-token matches can be measured from a sample of text with no model at all. In these runs, 24% or less on code and Wikipedia gave under 2 points, and so did pandas at 38%. The share is a cheap first filter and not a rule, since the notes at 20% gained 11 points and sympy at 35% gains little once the model reads more (next subsection). Small tables raised the loss slightly even with trust trained on the same kind of text (sympy 1.708 to 1.725 at 10% of the table and 1.755 at 1%; Wikipedia 3.126 to 3.133 and 3.144), and so did the table from another library (2.093 to 2.136). A table that raises the loss should be left off.

On Wikipedia the gain sits where several hint lengths agree. Where all four fire, 10% of places,

the top word rises from 48.5% to 57.2%; where three fire, 41.9% to 43.5%; with one or two the change is 0.5 points or less. It also sits where the model is unsure: 13.5% to 15.8% when its top word holds 10% to 25% of its probability, and 91.5% to 91.6% when that word holds over 90%.

13.3 More context removes most of the gain on ordinary code

A real assistant reads more than 128 tokens, so we repeated two code rows with the model reading 512 tokens and scored the last 120 places of each block (5,000 to 8,000 places). In sympy the model alone rose from 64.4% to 79.1%, and hints added only 0.8 points to reach 79.9%, with loss unchanged (0.904 and 0.907). In transformers unseen folders the model rose from 66.2% to 77.1%, and hints still added 8.6 points to reach 85.7% (loss 0.993 to 0.646). So on ordinary code, reading more text gives far more than a table does; only heavily copied code keeps a clear gain.

13.4 Whole phrases and cost

Single next words overstate a user's gain. Asking for 5 tokens in a row, 96 prompts each, the transformers split rose from 25 to 40 right, and sympy stayed level at 26 and 25.

A lookup took 0.66 ms per token in Python on the laptop, against 5.6 ms per token for the model reading 128 tokens at once. A table of 6M tokens took 234 MB of arrays and about 5 s to build, so a repeated text source can be added or rebuilt without touching the model.

13.5 A finished model gains little on general text because exact matches are rare

The model does not already know this text: its top word is right only 41.7% of the time on unseen Wikipedia. The gain is small because long exact matches are rare. Where all four hint lengths fire, 10% of places, the top word gains 8.7 points, which is most of the 1.4 point total. Over training, the loss gain also shrinks as a model learns. The width-40 model gained 0.63 loss at 90 steps, 0.52 at 360 and 0.34 at 1,440, and contexts it had never seen gained 0.42 per token against -0.03 for contexts seen 64 or more times (Table 16). The finished model gains 0.06 on Wikipedia. The comparison is rough: the text, table size and hint place all differ. Hints lower the loss most early in training, and at inference for repeated text that the model has not learned. The early drop is recall and not learning, because those models score worse than models trained with no hints when the hints are off.

14 Limits

- SetGram was tested on templated rewordings of made-up facts. The key uses token identity, so a synonym ("birthplace" for "born") or another language gives a different key; real paraphrase benchmarks are untested.
- SetGram keys collide: "x y z" and "z y x" share a key, and other facts with the same rare words give wrong hints. In the made-up language the table itself was right on 88% of keys; real text has far more collisions.
- The rare words must lie within the 16 tokens before the predicted token, so facts whose subject is far from the answer are missed.

- The baselines are simplified re-implementations at a small scale, not the published systems. Engram's case compression, multi-head hashing and scale, and the memory layer's size and placement, may change their numbers, though not the fact that their keys depend on the exact last tokens or on training.
- The SetGram scaling covers 4 times the weights and 4 times the steps, below the 8 times we aim for.
- The web text models stop at a loss far above real models, so they do not show what hints do for a trained model. The largest joint model has 4.3M parameters.
- Each setting was run once, and learning rates were picked on the same validation chunks used for reporting.
- Reasoning in this paper means one comparison task in a made-up language, plus held-out sums. It does not stand for reasoning in general.
- We did not run hints from earlier steps on the comparison task, so we do not know whether they keep it.
- The speedrun record measures time to a fixed early loss. It says nothing about final quality, and neither do our runs at equal steps.
- On the finished model, near copies between files remain in the table and test, the fact test has only 11 facts, and the model is 4-bit.
- Shakespeare is not web text: it has no crawled duplicates and its tokens are characters, so our result there does not contradict the blog post's rising returns.
- In a real run, leave-one-out needs the full corpus before training starts, or a table large enough to subtract one count per lookup.
- We measured loss at equal steps, not clock time, except where stated.

15 Open Questions

1. If a 1B-parameter model is trained with an exact n-gram table plus a SetGram table, does it answer reworded factual questions (for example paraphrased PopQA or biography benchmarks) better than with the exact table alone, at the same web loss?
2. If Engram's hashed embeddings are keyed by sets of rare words instead of the last 2 and 3 tokens, does it keep its benchmark gains and add reworded-fact recall?
3. If SetGram keys are built from word groups (stems, synonyms, or clusters of embeddings) instead of token identity, do paraphrases with different words also match?
4. If a gate is taught to predict which table, exact or SetGram, has the right next token, does it remove the drop when both are used in training?
5. If the trust of SetGram hints is learned on facts rather than on web text, do count-1 hints about facts held only in the table become usable in training?

6. If a finished large model reads code with a table of that code, does a user accept more completions than with search over the same code?

7. If a GPT-2-sized model is trained with Muon on web text, does adding n-gram hints from the training data lower validation loss compared with the same model without hints?

8. If a model is trained on web text with a hint corpus about 500 times larger than the tokens it trains on, does leaving out the token's page and dropping half of the hints beat hints from earlier steps?

9. If a model is trained on web text with hints, does each doubling of the hint corpus lower validation loss by more than the doubling before it, after near-duplicate documents are removed?

10. If a model is trained with hints, does it repeat training text word for word more often than the same model without hints?

11. If a model trains far longer than our web text runs, does the validation loss gain from short matches disappear?

12. If a GPT-2-sized model is trained on web text with a switch taught to predict whether each hint is right, is its validation loss lower than with plain hints, and does the lead keep growing with training length?

13. If a hint table holds passages the model trains on 15 or more times, does a model trained on web text lose reasoning skills that it keeps with 5 or fewer? Why does the hint at the word between two statements start the block in the toy?

14. If a model is trained on web text and reasoning tasks with 10% to 50% of the hint table and tested with all of it, does it keep comparison and sums, and how much of the web text gain does it keep?

16 Conclusion

Every memory we surveyed that stores a corpus and reads it inside the model looks up the last tokens in their exact order, so it cannot find a fact asked in other words. Keying the same next-token table by the set of rare words fixes this: SetGram recalled 80% to 96% of reworded facts, including facts the model never trained on, where exact hints, Engram-style n-grams, CoCoNTs-style targets, a memory layer and test-time exact mixing stayed at chance. It adds little to web text loss on top of exact keys, so we recommend it as a second table beside an exact one, read at test time or through a gate that learns which table to trust.

For hints from a full corpus, remove each token's own occurrence, and on web text remove its whole page. On Shakespeare, dropping half of the hints as well beat hints from earlier steps by about 0.08 at three sizes, and by more when the corpus was far larger than the tokens trained. On web text with the page left out, dropping hints did not help (7.825 against 7.816). If a run makes more than one pass, causal hints need the same removal. These steps fix validation loss only: a full table in training still stops the model from memorizing and from learning comparison. If hints must not replace the model's own learning, the simplest candidate is a small table in training and the full table at test time, which is the record's own choice. It kept comparison and improved facts in our small test but is untested on web text. With test counts scaled, the limit there was about 5 copies of each training sentence in the table, not table size; without scaling it did not hold at

14 names. The tested option on web text is to train every step with and without hints and add a switch trained to predict whether each hint is right. In our joint test this restored comparison with hints on and lowered web text loss further at the longest training, though sums stayed blocked with hints on. Hints at the output only need no second pass and keep reasoning, at the cost of part of the gain. These are small models and single runs, so all of this must be tested in real training on FineWeb.

On a finished model, add hints only for text that repeats word for word and that the model has not learned. The share of exact 6-token matches is a cheap first filter, not a rule. Then measure loss with and without the table on held-out text at the context length you will use, and leave off any table that raises it. On general text the largest loss drops came early in training, and they are recall and not learning, because those models score worse with hints off than models trained with no hints.

References

[1] H. Brunborg. N-gram hinted language modeling: NanoGPT Speedrun in 6.7s. `https://brunborg.com/n-gram-hinted-lm`, 2026.

[2] Track 1: Retrieval-augmented n-gram hints. Pull request 367, `https://github.com/KellerJordan/modded-nanogpt/pull/367`.

[3] H. Brunborg. n-gram-hinted-lm, file `old_records/online_hints.py`. `https://github.com/hermabr/n-gram-hinted-lm`.

[4] K. Jordan. Muon: an optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

[5] Muon is scalable for LLM training. arXiv:2502.16982, 2025.

[6] Wang et al. Muon outperforms Adam in tail-end associative memory learning. arXiv:2509.26030, 2025.

[7] Conditional memory via scalable lookup: a new axis of sparsity for large language models. arXiv:2601.07372, 2026.

[8] Tensorizing Engram: sharing latents across n-gram embeddings is beneficial in LLMs. arXiv:2606.08347, 2026.

[9] Lngram v2: latent n-gram memory with interpretable discrete representations. arXiv:2609.03426, 2026.

[10] Improving language models by retrieving from trillions of tokens. arXiv:2112.04426, 2021.

[11] D. Micci-Barreca. A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. SIGKDD Explorations, 2001.

[12] L. Prokhorenkova et al. CatBoost: unbiased boosting with categorical features. NeurIPS, 2018.

[13] N. Srivastava et al. Dropout: a simple way to prevent neural networks from overfitting. JMLR, 2014.

[14] V. Rosić. Muon lab. `https://github.com/vukrosic/muon-lab`, 2026.

[15] N. Kandpal et al. Large language models struggle to learn long-tail knowledge. ICML, 2023.

[16] Tracing multilingual factual knowledge acquisition in pretraining. arXiv:2505.14824, 2025.

[17] Positional fragility in LLMs: how offset effects reshape our understanding of memorization risks. arXiv:2505.13171, 2025.

[18] Memory vs. context? Influential factors of factual recall in language models. arXiv:2609.24238, 2026.

[19] Generalization through memorization: nearest neighbor language models. arXiv:1911.00172, 2019.

[20] Long-tail crisis in nearest neighbor language models. arXiv:2503.22426, 2025.

[21] X. Wang et al. Generalization v.s. memorization: tracing language models' capabilities back to pre-training data. ICLR, 2025. arXiv:2407.14985.

[22] T. Norlund et al. On the generalization ability of retrieval-enhanced transformers. arXiv:2302.12128, 2023.

[23] V. Rosić. ngram-hints-llm: code for this paper. `https://github.com/vukrosic/ngram-hints-llm`, 2026.

[24] kNN-Adapter: efficient domain adaptation for black-box language models. arXiv:2302.10879, 2023.

[25] NeuML. wikipedia-20260401. `https://huggingface.co/datasets/NeuML/wikipedia-20260401`.

[26] SpecDec++: boosting speculative decoding via adaptive candidate lengths. arXiv:2405.19715, 2024.

[27] S. Ravfogel, G. Yehudai et al. Geometric Factual Recall in Transformers. arXiv:2605.12426.

[28] Privileged Information Dropout in Reinforcement Learning. arXiv:2005.09220.

[29] X. Liang, L. Wu et al. R-Drop: Regularized Dropout for Neural Networks. NeurIPS 2021.

[30] J. Liu, S. Min, L. Zettlemoyer, Y. Choi, H. Hajishirzi. Infini-gram: scaling unbounded n-gram language models to a trillion tokens. COLM, 2024. arXiv:2401.17377.

[31] A. Sathe, S. Sarawagi. Efficient training of language models with compact and consistent next token distributions (CoCoNTs). arXiv:2407.02819, 2024.

[32] Z. Allen-Zhu, Y. Li. Physics of language models: part 3.1, knowledge storage and extraction. ICML, 2024. arXiv:2309.14316.

[33] H. Chang et al. How do large language models acquire factual knowledge during pretraining? arXiv:2406.11813, 2024.

[34] L. Berglund et al. The reversal curse: LLMs trained on "A is B" fail to learn "B is A". arXiv:2309.12288, 2023.

[35] R. Kneser, H. Ney. Improved backing-off for m-gram language modeling. ICASSP, 1995.

[36] R. Rosenfeld. A maximum entropy approach to adaptive statistical language modelling. Computer Speech and Language 10:187–228, 1996.

[37] R. Kuhn, R. De Mori. A cache-based natural language model for speech recognition. IEEE TPAMI 12(6), 1990.

[38] Y. Qu et al. NGM: a plug-and-play training-free memory module for LLMs. arXiv:2605.16893, 2026.

[39] R. Cheng et al. Memory Grafting: scaling language model pre-training via offline conditional memory. arXiv:2605.20948, 2026.

[40] Y. Ma et al. TF-Engram: a train-free Engram with SSD-backed memory for large language models. arXiv:2607.07388, 2026.

[41] B. Yang et al. FactorEngram: factorized n-gram memory with basis-level gating for language models. arXiv:2609.35578, 2026.

[42] Y. Chen et al. Beyond n-gram: data-aware X-GRAM extraction for efficient embedding parameter scaling. arXiv:2604.21724, 2026.

[43] A. Roy et al. N-Grammer: augmenting transformers with latent n-grams. arXiv:2207.06366, 2022.

[44] H. Huang et al. Over-tokenized transformer: vocabulary is generally worth scaling. arXiv:2501.16975, 2025.

[45] D. Yu et al. Scaling embedding layers in language models (SCONE). arXiv:2502.01637, 2025.

[46] G. Lample et al. Large memory layers with product keys. NeurIPS, 2019. arXiv:1907.05242.

[47] V.-P. Berges et al. Memory layers at scale. ICML, 2025. arXiv:2412.09764.

[48] Y. Wu et al. Memorizing transformers. ICLR, 2022. arXiv:2203.08913.

[49] O. Ram et al. In-context retrieval-augmented language models. TACL, 2023. arXiv:2302.00083.